

Towards Automatic and Efficient Prediction Query Processing in Analytical Database

Yuchen Peng, Zhongle Xie*, Ke Chen*, Gang Chen, Lidan Shou
 The State Key Laboratory of Blockchain and Data Security, Zhejiang University
 {zjupengyc,xiezl,chenk,cg,should}@zju.edu.cn

Abstract—Data analysts nowadays are keen to have analytical capabilities involving deep learning (DL). Prediction queries, which combine relational operations with DL models to analyze multi-modal data, provide a powerful facility for smart in-database analysis. However, loose integration systems, which support such queries via User-Defined Functions (UDFs) and external runtimes, often impose high economic costs, particularly in cloud-based environments; while tight integration systems, which implement model inference through a sequence of explicitly written SQL queries, incur heavy user burdens and huge optimization space.

In this paper, we introduce PEPS, an end-to-end analytical database for automatic and efficient prediction query processing. PEPS automates the process of prediction query synthesis and ensures usability through declarative schemes. Additionally, it improves query performance with offline optimization using the DB-oriented Computation Graph Optimization (DBCBO) algorithm and online optimization via heuristic query rewriting. Empirical evaluations show that PEPS offers better usability than baseline methods, provides lower economic costs, and achieves performance speedup compared to advanced Python UDFs.

Index Terms—Query Processing, Query Optimization, In-database Inference

I. INTRODUCTION

With the rapid progress of artificial intelligence (AI) in many applications such as image classification and natural language processing, data analysts are increasingly leveraging this technology within critical decision-making processing [1]–[4]. In response to the demands of complex analytical tasks, substantial efforts are made in the database community to develop in-database deep learning (in-DB DL) techniques [5]–[8]. In real-world applications, numerous analytical database vendors, including Databricks [9], Google BigQuery [10], and PolarDB [11], facilitate model inference capabilities primarily powered by DL models, via proprietary APIs to support complex analytics.

A typical scenario for leveraging DL models involves a query that integrates relational operators with DL inference, commonly referred to as a *prediction query* [7], [12]. Fig. 1 demonstrates a prediction query used to count the number of red objects in photos taken during 2023. Specifically, `Object()` and `Color()` represent the processes of DL inference for object and color classification tasks.

Generally, the implementation of in-DB DL inference is categorized as either *loose integration* or *tight integration*, as illustrated in Fig. 1. Loose integration solutions typically

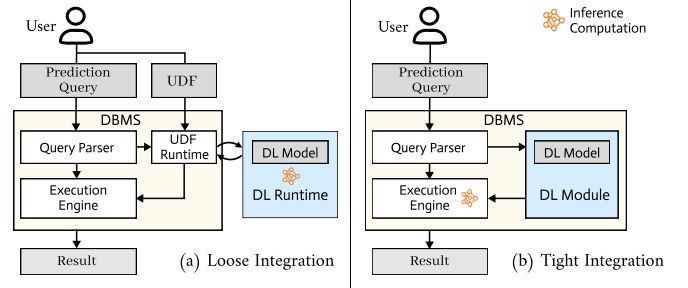


Fig. 1: Comparative workflow of in-DB DL inference.

```
SELECT Object(I.image) AS O, count(F.id) AS C
FROM FABRIC F, IMAGES as I
WHERE F.createDate >= '2023-01-01'
      and F.createDate <= '2023-12-31'
      and Color(I.image) = 'red'
GROUP BY O
```

Listing 1: A prediction query of image analysis task.

utilize User-Defined Functions (UDFs) [5], [13], [14] to invoke DL inference, with data exchanged using formats such as Arrow [15] or Parquet [16]. UDF solutions provide simplified integration and support hardware acceleration through tensor computation runtimes [17]–[21]. However, in typical cloud-based analytics scenarios, economic costs [22] are considered equally, if not more, significant as usability for end users. As each UDF invocation requires substantial computational resources, users with limited budgets might find that the high cost of these invocations emerges as the primary performance bottleneck for prediction queries.

In contrast, tight integration solutions store DL models as relational tables and implement DL inference via a sequence of explicitly written SQL queries, termed *mQueries* (abbr. for model queries). In this way, the DBMS processes inference requests by executing mQueries alongside the model parameters stored in relational tables [7], [23]–[25], eliminating data exchange required by loose integration solutions. Furthermore, this approach enables fine-grained resource control, allowing for query optimization based on data access patterns. For instance, SQL-based compute-intensive analytics can outperform Numpy due to superior data locality and vectorization [24].

Table I summarizes the key features of the loose and tight integration designs. From an end-user perspective, tight integration approaches offer notable advantages over loose integra-

* Corresponding authors

TABLE I: Comparison between the two designs of in-DB DL inference.

	<i>Inference Engine</i>	<i>Usability</i>	<i>Economic Costs</i>	<i>Implement. Efforts</i>	<i>Performance Opt. Space</i>
Loose Integration	external runtime	high	high	limited	limited
Tight Integration	DBMS	low	low	large	large

TABLE II: Detailed task breakdown of UC09 in the TPCx-AI benchmark (numbers were collected from a user study).

	Data Loading	Feature Extraction	Model Profiling	Data Preparation	Query Composition	Query Evaluation
Lines of Code	10	56	15	206	750	—
Manpower (Est.)	0.1h	0.5h	0.5h	1h	3h	—
Execution Time	3.47s	185.78s	128.52s	309.08s	—	318.34s

tion, particularly in terms of economic efficiency and potential performance improvements. However, despite the tremendous efforts in implementation, tight integration approaches face two major challenges in usability and performance during the process of query synthesizing and query processing:

Challenge 1: Heavy user burden. Users employing tight integration approaches must engage in complex coding tasks, such as parsing the DL model structure and composing prediction queries. Table II presents the task breakdown for a face recognition pipeline using ResNeXt [26] in the TPCx-AI Benchmark [27], with data collected through a user study involving 8 participants with backgrounds in both database systems and DL. The substantial amount of code required in these steps demands considerable effort, even from experts, let alone from database administrators to write error-free DL-related code. To alleviate user burden, techniques for automatically synthesizing/composing mQueries might be needed.

Challenge 2: Non-trivial performance optimization. Performance optimization remains a significant challenge in tight integration approaches [6], primarily due to two points. First, DBMS lacks native support for optimizing mQueries [8], as the relational data model is ill-suited to handle large tensors [25]. To mitigate the inferior performance of complex prediction queries, users even have to manually rewrite their queries with pruning techniques such as pre-join [7] and predicate reordering [28]. Second, the existing DL acceleration techniques are not readily applicable for in-DB settings. While graph substitution [29], [30] is the most common and effective method, it requires estimating the cost of actual deployment time on hardware, commonly GPU, making it incompatible with CPU-based database execution engines.

To address the challenges mentioned above, we present PEPS, a **P**rediction qu E ry **P**rocessing **S**ystem. PEPS offers declarative interfaces for defining and manipulating data for model inference, enabling users to easily synthesize queries while the system autonomously preprocesses data and synthesizes queries. It extends SQL with a specialized data definition language (DDL) to convert unstructured data into structured arrays, allowing seamless integration of both structured and

unstructured data with automated feature extraction.

PEPS builds upon user-defined operators (UDOs), namely embedded UDFs on multi-dimensional arrays (MDA), to better handle the data for model inference, advancing the processing against existing tight integration solutions [7]. To tackle the optimization challenge associated with prediction queries, we introduce techniques during both the offline and online processing phases: (1) In the offline processing phase, we propose the *DB-oriented Computation Graph Optimization* (DBC GO) algorithm to refine DL models and optimize computation graphs to reduce query complexity and storage overhead. (2) In the online processing phase, we present heuristic query rewriting on query plans and a DL-aware optimizer to improve execution efficiency via mQuery inlining.

Leveraging the designs above, PEPS enables end users to simply launch prediction queries, while shielding them from the complexities of query synthesizing and performance optimization. In summary, we make the following contributions:

- We design and build an end-to-end analytical database, PEPS, which firstly automates the process of prediction query synthesis. With its declarative scheme for in-DB DL inference and optimization components, it ensures usability for users and efficiency for prediction query processing (Section III).
- We propose a novel DB&DL cost model to precisely evaluate the cost of in-DB DL inference. Based on this, we present, for the first time, a model computation graph optimization algorithm to reduce the computational complexity of DL inference inside DBMS (Section IV).
- We further enhance PEPS by developing a two-phase online optimization workflow with novel transformation rules to improve the holistic prediction query performance (Section V).
- We experimentally demonstrate that, compared to the loose integration methods, PEPS achieves up to $70\times$ speedup for simple prediction queries and up to $40\times$ speedup for complex prediction queries (Section VI).

The code for PEPS is open-sourced at the following link: <https://github.com/zjuDBSystems/PEPS>.

II. PRELIMINARIES

In this section, we first define the process of DL inference and then present two representations enabling in-DB DL inference. We finally present the definition of prediction query and the in-DB inference system.

DL Inference. Given input data $\mathcal{X}$, DL inference is the process of using a trained DL model $\mathcal{M}$ to produce a confidence coefficient or a classification label as a prediction result, denoted as $\mathcal{Y}$.

Particularly, $\mathcal{M}$ consists of m layers, represented as $\mathcal{M} = \langle l_1, l_2, \dots, l_m \rangle$. Each layer l_i implements a computation function $\sigma_i(\mathbf{t}_{i-1}; \Theta_i)$, where $\mathbf{t}_{i-1}$ is the input tensor and Θ_i is the trained parameter tensor of the layer. Note that the parameters, also referred to as the weights of l_i , are expressed as a parameter tensor (i.e., a multi-dimensional array). During the inference process, the first layer (known as the input layer)

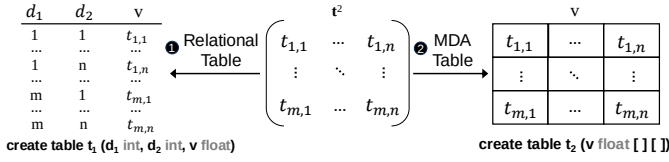


Fig. 2: Alternative schemes for storing tensors as tables.

receives the input data $\mathcal{X}$ as an input tensor $\mathbf{t}_0$. Subsequently, each layer l_i takes $\mathbf{t}_{i-1}$ as input and outputs a tensor $\mathbf{t}_i$, computed as $\mathbf{t}_i = \sigma_i(\mathbf{t}_{i-1}, \Theta_i)$. The final layer (known as the output layer) produces $\mathbf{t}_m$ and applies a specific activation function to $\mathbf{t}_m$, thereby generating the inference result $\mathcal{Y}$.

To enable in-DB DL inference, computations are performed using relational tables and relational queries [7]. To this end, we introduce the following two representations.

Tensors as Relational Tables. Tensors are converted into relational tables within the database. The conversion process is illustrated in Fig. 2. For a k -dimensional tensor $\mathbf{t}^k$, each value is denoted as $t_{i_1, i_2, \dots, i_k}$, where each index i_q specifies the position along the q -th dimension. To store $\mathbf{t}^k$ as a relation, a relational schema $\mathcal{R}^t = (d_1, d_2, \dots, d_k, v)$ is defined, where each dimension attribute $d_q (1 \leq q \leq k)$ corresponds to the mapping $d_q \mapsto i_q$ and the value attribute v corresponds to the mapping $v \mapsto t_{i_1, i_2, \dots, i_k}$. The cardinality of $\mathcal{R}^t$ equals the number of values in $\mathbf{t}^k$, and each value v is accessed by specifying the corresponding indexes.

With this transformation, the model $\mathcal{M}$ is stored in the DBMS by converting the parameter tensor Θ_i of each layer into a relational table, denoted as R^{Θ_i} . Note that the input tensor of each layer is generated dynamically during the inference process.

DL Operations as Relational Queries. With the parameter tensors maintained as relations, the computation functions in each layer of $\mathcal{M}$, namely the DL operations, are processed using standard relational queries [31]. Specifically, σ_i of each layer is first transformed into relational algebra [31]–[33] and then rewritten as a relational query, denoted as q^{σ_i} . In this way, a sequence of SQL queries is generated by concatenating q^{σ_i} in the order of the layers of $\mathcal{M}$. We introduce *mQuery* as a new SQL syntax to wrap these SQL queries, defined as follows.

DEFINITION 1 An *mQuery* $mQ = \{q^{\sigma_1}, q^{\sigma_2}, \dots, q^{\sigma_i}\}$ is a sequence of SQL queries. For any given input tensor $\mathbf{t}_0$ generated by the input data $\mathcal{X}$, the output $\mathcal{Y}$ of running $\mathcal{M}$ is equivalent to the result obtained by cascaded executing queries in mQ on the relational form of $\mathbf{t}_0$, i.e., $\mathcal{R}^{\mathbf{t}_0}$.

In the definition, “cascaded executing” refers to the process of executing the SQL queries in mQ where each query q^{σ_i} takes the output of the preceding query $q^{\sigma_{i-1}}$ as input and produces output to the next query $q^{\sigma_{i+1}}$.

The usage of R^{Θ_i} and *mQ* enables DL inference within the DBMS execution engine, relying solely on SQL queries and relational tables. In practical analytical scenarios, the inference is often combined with conventional SQL statements to formalize complex queries. We call such queries prediction queries, which are defined as follows:

Prediction Query. Given a dataset $\mathcal{D}$ and a set of DL models $\mathbb{M}$, a prediction query $\mathcal{PQ}$ is a complex query that integrates one or multiple in-DB DL inference(s), where each applies an invocation to a model in $\mathbb{M}$, into an underlying relational query to gain the analytical insights from $\mathcal{D}$.

We call the system natively supporting prediction queries in DBMS as **in-DB inference system**. Similar to the implementation of DL inference, the execution of prediction queries is achieved by integrating mQueries and corresponding relational tables storing tensors into the DBMS execution engine. Thus, the design principle of the in-DB inference system, though involving heavy engineering workloads, urges minimal changes to the original DBMS kernel and minimal overhead from inference modules.

III. SYSTEM OVERVIEW

In this section, we provide an overview of PEPS, an in-DB inference system designed to facilitate efficient and automatic prediction query processing.

A. Data Model

A key challenge of in-DB inference systems is managing both structured (e.g., tabular) and unstructured (e.g., image, text) data. PEPS addresses this by introducing *collaborative relation*, which extends the relational paradigm by handling unstructured data as tensors, enabling efficient storage and precise cardinality estimation.

DEFINITION 2 A *collaborative relation* R^c is a relation with $n > 1$ attributes $A_1, \dots, A_n$, where each attribute A_i is either a standard relational attribute or an unstructured attribute. Additionally, R^c must have at least one relational attribute.

PEPS extends the data definition language (DDL) in SQL with self-defined syntax for creating collaborative relations, abbreviated as `cTABLE` and illustrated in Listing 2. To deal with unstructured attributes, an additional keyword `TRANSFORM` is required, followed by a primitive transformation function. Formally, given the unstructured attribute A_i^f , the transformation function e is defined as a mapping satisfying $e : I \rightarrow U$, which converts input unstructured data $I \in A_i$ to a tensor U . Note that five common transformation functions [34], including `Normalize`, `Standardize`, `Encode`, `Scale`, and `Impute`, are provided by default.

To achieve better performance, PEPS employs a special data type, i.e., multi-dimensional array (MDA) in SQL:2023 [35], to store the tensors. The motivation is that compared to relational representations discussed in Section II, MDA offers two advantages: (i) it reduces redundant dimension index storage [7], (ii) exhibits superior computational performance in terms of tensor computations [24]. More importantly, the MDA-based representation enables precise cardinality estimation of tensor operations, which is crucial for query optimization of in-DB DL inference.

Fig. 3a shows a query used to create a collaborative relation named `CIFAR`. The annotated function `normalize` normalizes the image pixel values as illustrated in Fig. 3b, resulting in the MDA

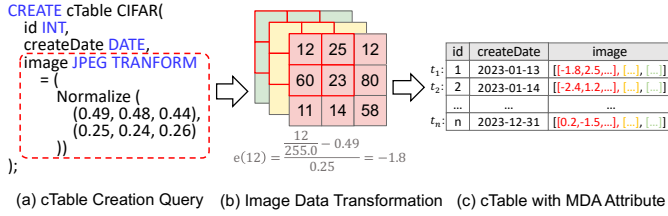


Fig. 3: An example of collaborative relation management.

```
CREATE cTABLE name (
  [[attributeName attributeType
    [TRANSFORM
      <Normalize|Standardize|Encode|Scale|Impute>
    ]],
  [...]]
)
```

Listing 2: The syntax of collaborative relation creation.

attribute stored in the database as shown in Fig. 3c. The supported new data types include JPEG and PNG for images and TEXT for strings.

Discussion. The usage of the transformation functions, as well as TRANSFORM keyword, enable the native support of database systems running prediction queries. In particular, the functions automate the feature extraction process by converting unstructured data into tensors during data loading, eliminating the need for manual preprocessing shown in Table II. Consequently, the user has a more friendly experience in querying, which is the premier motivation of the design of PEPS.

B. Query Model

Beyond traditional SQL queries, PEPS supports prediction queries defined in Section II. Unlike existing in-DB inference systems requiring users to manually construct complex prediction queries, PEPS extends the SQL query model, enabling users to declare and specify DL inference directly within SQL queries. Particularly, *inference functions*, defined as *nUDFs*, are introduced with a straightforward syntax as detailed below.

DEFINITION 3 An inference function (*nUDF*) is a scalar function $f : x \xrightarrow{mQ^M} y$ that takes a value x in an unstructured column as input and invokes the processing of *mQuery* corresponding to a DL model M , denoted as mQ^M , on x to output a confidence coefficient or a classification label y .

PEPS extends SQL with the INFERENCE FUNCTION keyword for *nUDF* creation, as illustrated in Listing 3, where a model file path specifies which DL model to be loaded. Internally, PEPS represents *nUDFs* as *mQueries* to enable integrated optimization and evaluation with other relational operators. We formulate a prediction query as $PQ = (Q_{db}, \mathcal{F})$, where Q_{db} represents relational operators and $\mathcal{F}$ refers to in-DB DL inferences. To evaluate PQ , PEPS first synthesizes an *mQuery* for each *nUDF* and combines them with Q_{db} to form the final query. PEPS then rewrites and optimizes PQ

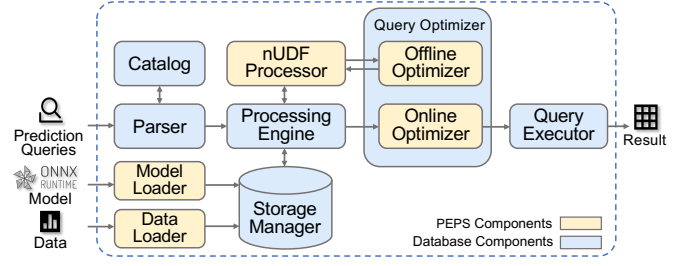


Fig. 4: The architecture of PEPS.

```
CREATE INFERENCE FUNCTION Color(JPEG image)
FROM MODEL IN FILE 'resnet.onnx'
```

Listing 3: An *nUDF* creation query.

for runtime performance, which is detailed in Section V. This process is transparent to users, obviating the need for *mQuery* composition and optimization from the user side.

Discussion. The design of *nUDFs* offers three core benefits targeting prediction queries. First, the declarative interface eliminates the complexity of *mQuery* composition, alleviating the user burden of manual model parsing. Second, by implementing *nUDFs* as extended database operators, PEPS enables seamless integration of standard SQL operations with in-DB DL inference in prediction queries. Finally, as a dedicated operator, *nUDFs* allow PEPS to apply specialized optimization techniques during query optimization.

C. System Architecture

Fig. 4 depicts the system architecture of PEPS. We follow a minimal viable principle when designing PEPS. The yellow components in the figure are attached to the original database modules in blue to deal with new syntaxes defined in the data model and query model.

PEPS extends a relational database with three components: *Data Loader*, *Model Loader*, and *nUDF Processor*. The *Data Loader* ingests raw data and transforms it into collaborative relations. The *Model Loader* manages DL models and extracts their computation graphs. Leveraging this component, the *nUDF Processor* coordinates *nUDF* creation and optimization.

For performance optimization, PEPS further augments query processing engine with two components: *Offline Optimizer* and *Online Optimizer*. The *Offline Optimizer* optimizes the computation graphs of the models directly in databases before invoking the inference process. Details are presented in Section IV. The *Online Optimizer* adjusts the execution plan of prediction queries at runtime, including both relational operators and *nUDFs*, to improve overall performance. The optimization strategies are elaborated in Section V.

IV. OFFLINE PROCESSING

In the offline stage, PEPS instantiates the *nUDF* and loads the model with corresponding optimizations to improve the performance of in-DB DL inference. Fig. 5 illustrates the workflow of offline processing. The *nUDF Processor* first

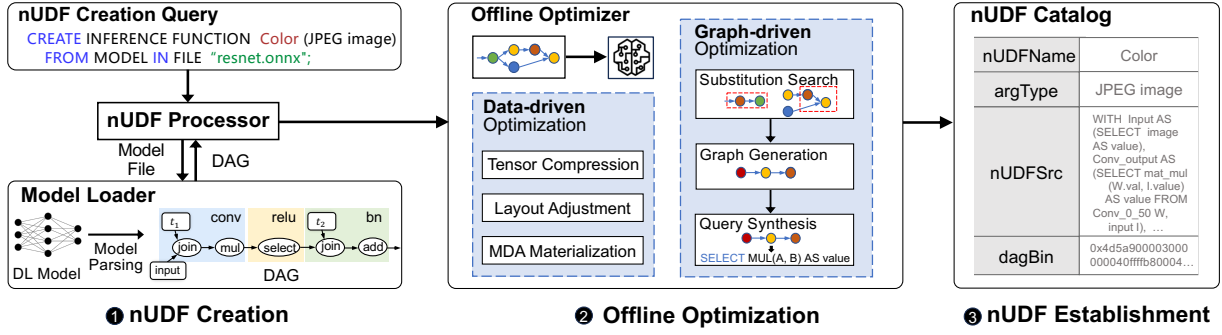


Fig. 5: Overview of the offline processing workflow.

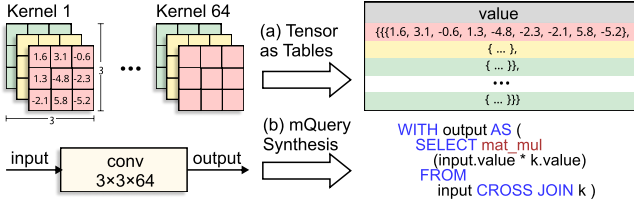


Fig. 6: An illustration of Model Loading in PEPS.

parses the creation query for the nUDF and passes the model file into the *Model Loader* to generate a directed acyclic graph (DAG) of relational operators, which serves as the computation graph of the given model. The *Offline Optimizer* applies *graph-driven* and *data-driven* optimizations on the computation graph and the model parameters, respectively. The final output of this process, stored as the nUDF’s metadata, consists of the optimized parameter tables and the mQuery corresponding to the refined computation graph. This entire process is automated in PEPS to minimize the manual intervention required for users when loading a model into the database.

A. Model Loading

PEPS supports loading DL models represented in ONNX [36] or other compatible formats. A DL model consists of two parts: the trained parameters and a computation graph that defines DL operations. The Model Loader deals with the two parts separately.

1) *Trained Parameters*: Upon extracting model parameters as tensors, the Model Loader utilizes the data type of MDA to store each tensor as an array table. The top half of Fig. 6 illustrates this process for the parameters in a convolution layer. In the figure, the parameter (i.e., convolution kernel) is represented as a 3-dimensional array, with dimensions corresponding to height, width, and channels, respectively. In this way, PEPS organizes all trained parameters of the model within array tables. Each table contains a single MDA attribute dedicated to its corresponding tensor.

2) *Computation Graph*: The *Model Loader* traverses the computation graph in topological order and converts it to a corresponding DAG in the database. Each node in the computation graph, representing a DL operation, is mapped to a series of operators to enable the inference computation within the database. To effectively support MDA computations, we introduce a suite of fundamental user-defined

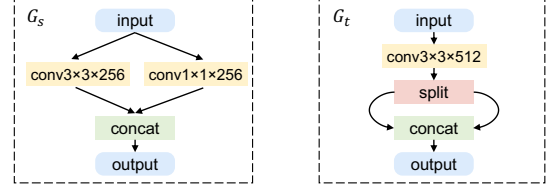


Fig. 7: A typical substitution rule s .

operators (UDOs), which are fine-grained, built-in UDFs that can be used as algebraic operators to extend the relational algebra utilized by existing DBMS [37]. Our implementation includes fundamental matrix operations, such as `mat_mul` (matrix multiplication), `eadd` (element-wise addition), and `max_pool` (max pooling).

For instance, a convolution operation is transformed to the matrix multiplication between the MDA data in the input and parameter tables, as shown at the bottom of Fig. 6. Parts of the mQuery for such operation are composed using operators of `select`, `join`, and `mat_mul`.

nUDFs are maintained through catalog tables, as depicted in Fig. 5. The associated metadata includes the nUDF name (`nUDFName`), the nUDF arguments (`argType`), the underlying serialized DAG (`dagBin`), and the synthesized mQuery (`nUDFSrc`). Following the loading process, invocations of the nUDF are substituted by mQueries, which is equivalent to the DAG containing corresponding relational operators and UDOs within the database.

Discussion. Relational operators are ill-suited for complex MDA manipulations [38], incurring higher storage costs and increased computational overhead due to substantial intermediate results [7]. Therefore, we sophisticatedly design the fundamental UDOs without forcing frequent flattening or reconstructing arrays, thereby ensuring higher computation throughput. Compared to existing in-DB inference systems [6], [7], PEPS automates the transformation of DL models into database-friendly DAGs and mQueries, enabling subsequent graph-based optimizations. In addition, leveraging parallel query processing, PEPS schedules UDOs and relational operators in a pipelined fashion, allowing data to flow seamlessly through stages without materialization.

B. Graph-driven Optimization

Once a DL model has been loaded into PEPS, further performance improvements can be achieved by optimizing

the computation graph before online processing. Existing DL systems [39], [40] typically rely on measuring each DL operator's execution time in a standalone DL runtime environment and applying rule-based subgraph substitutions to reduce cost. However, such optimizations are not supported natively in databases due to the differences in the runtime environment. To bridge this gap, we propose *DB-oriented Computation Graph Optimization* (DBC GO) algorithm. The algorithm integrates a backtracking search algorithm tailored to in-DB inference, locating beneficial substitutions in the computation graph based on a *database-specific* cost model.

1) *Optimized Computation Graph Searching*: We use pre-defined substitution rules to replace subgraphs of the computation graph with more efficient alternatives. Although such substitutions can reduce execution costs, they introduce a large combinatorial search space up to $O(|S|!)$, where S is the set of rules, making brute-force search impractical. To address this, DBC GO employs backtracking to prune the search space. The pseudocode of the algorithm is presented in Algorithm 1. It takes as input a computation graph $G_{in} = (V, E)$, and a set of substitution rules S , where V and E denote the sets of nodes and edges, respectively. A priority queue $\mathcal{P}$ is initialized with G_{in} and used to track candidate graphs (Line 1), dequeued in ascending order of their estimated processing cost, computed by the cost model discussed in Section IV-B2.

The search algorithm generates new graphs for each dequeued graph G by applying graph substitution rules $s \in S$ to G (Line 5). We formally define a substitution rule s as a function $G_s \xrightarrow{s} G_t$, where G_s and G_t denote the source and target graph, respectively. The substitution applies s to replace a matching subgraph in G with G_t . Fig. 7 shows a typical substitution rule, where two convolution operations are substituted by a new convolution operation followed by a split operation. We employ the 157 substitution rules from TASO [39] as the rule set S in PEPS. The correctness of these rules has been verified in the original paper.

As shown in Line 10, the newly generated graphs are enqueued for a new iteration of the graph substitutions. The process terminates when $\mathcal{P}$ is empty, at which point the final graph G_{opt} is returned. Note that we introduce a hyper-parameter, denoted as α , to balance the trade-off between the risk of overlooking potential recursive substitutions and the challenge of a vast search space that demands considerable processing time (Line 9). In practice, by setting α slightly greater than 1, the search algorithm can explore a broader graph optimization space than a simple greedy algorithm, while returning an optimized graph within a finite search time.

2) *DP-based Cost Model for the computation graph*: In DBC GO, the cost of the computation graph is accumulated by the cost of each node $v \in V$, following the graph's topological order. Each DL operation represented by v is transformed into a combination of standard relational operators and UDOs (see Section IV-A2). We denote this combination as $O_v = \{o_1, o_2, \dots\}$, where o_i is the i -th relational operator or UDO in the combination. Therefore, the cost of node v is determined by summing the costs of o_i along with the cost of

Algorithm 1 DB-oriented Computation Graph Optimization

Input: An initial computation graph G_{in} , a set of graph substitutions rules $\mathcal{S}$, and a DB-oriented cost model $Cost(\cdot)$.

Output: An optimized graph G_{opt} .

```

1:  $\mathcal{P} = \{G_{in}\}$ 
2: while  $\mathcal{P} \neq \{\}$  do
3:    $G = \mathcal{P}.dequeue()$ 
4:   for substitution rule  $s \in \mathcal{S}$  do
5:      $G' = s(G)$ 
6:     if  $Cost(G') < Cost(G_{opt})$  then
7:        $G_{opt} = G'$ 
8:     end if
9:     if  $Cost(G') < \alpha \times Cost(G_{opt})$  then
10:       $\mathcal{P}.enqueue(G')$ 
11:    end if
12:  end for
13: end while
14: return  $G_{opt}$ 

```

its predecessor nodes. Formally, the cost of node v , denoted as C_v , is calculated by:

$$C_v = \begin{cases} C_{O_v}, & \text{if } |v.prev| = 0 \\ C_{O_v} + \sum_{v' \in v.prev} C_{v'}, & \text{if } |v.prev| = 1 \\ C_{O_v} + \max_{v' \in v.prev} C_{v'} + C_{sync}, & \text{if } |v.prev| > 1 \end{cases} \quad (1)$$

where $v.prev$ is the set of predecessor nodes of v in the computation graph, and C_{O_v} is the total cost of all the operators and UDOs in the combination, calculated by $C_{O_v} = \sum cost(o_i)$. C_{sync} denotes the synchronization cost incurred when parallel processing the database operators representing the predecessor nodes.

The cost of relational operators is calculated using traditional estimation methods in databases, primarily relying on Disk I/O. In contrast, the cost of a compute-intensive UDO, denoted as o^u , is dominated by CPU usage rather than Disk I/O. To evaluate o^u , we first estimate the number of floating point operations (FLOPs) required to execute the operator, denoted as Φ_{o^u} . Next, we calculate the running time needed for these FLOPs and normalize this time to align with the units used for relational operators, ensuring consistency in cost estimation. Formally, the cost of o^u is estimated by:

$$cost(o^u) = \Phi_{o^u} * \frac{1}{\beta_{db}} * \frac{seq_scan_cost}{seq_time} \quad (2)$$

where β_{db} denotes the number of FLOPs per second that the DBMS can process, seq_scan_cost denotes the estimated I/O cost in disk page fetches, and seq_time is the execution time of a sequential scan measured through sampling on representative tables. Their ratio normalizes the cost of o^u to the cost units of the database.

To compute Φ_{o^u} , we define a function $\phi_{o^u}(U, W)$ for each o^u , where U and W denote the input and the parameter MDA, respectively. The output of $\phi_{o^u}(U, W)$ is determined by the

dimensions of U and W , facilitating rapid estimation without delving into specific operations within o^u .

The cost model in Equation 1, implemented through dynamic programming, classifies nodes into three types: (1) root nodes (no predecessor nodes), (2) trivial internal nodes (exactly one predecessor node), and (3) complex internal nodes (two or more predecessor nodes). For the first two types, the cost is simply the node's intrinsic cost or the cumulative cost passed from its predecessor. For complex internal nodes, however, the cost is determined by the most time-consuming predecessor plus a synchronization overhead, reflecting the parallel execution model in PEPS. While existing approaches [30] optimize inter-operator parallelism on GPU, they fail to account for the unique characteristics of database query processing. Instead, our approach uniquely adapts the parallel critical path model to database systems by leveraging *pipelined query execution*. This enables PEPS to optimize the computation graph by exploiting the potential overlap among database operators from distinct segments of the query plan. For instance, the branching operations in grouped convolutions are mapped to multiple independent subqueries, each dispatched to a separate CPU core, thereby increasing resource utilization and improving query performance.

The total cost of the computation graph is equal to $C_{v_{last}}$, where v_{last} is the last node in the topological order. In particular, our algorithm requires a single graph traversal to initialize the cost hashmap. During graph substitutions, only the affected subgraph starting from the substituted node is updated, avoiding a full re-traversal and reducing overhead.

3) *Complexity Analysis*: The algorithm's search space is bounded by the length of the priority queue, denoted as $|\mathcal{P}|$. Note that during graph substitutions, we update and record the hash values of the historical substitutions to avoid redundant substitution sequences. Therefore, $|\mathcal{P}|$ is bounded by $O(|V| \times |S|)$, where $|V|$ is the number of nodes in the original computation graph and $|S|$ is the number of substitution rules. Applying a rule s and calculating the cost of the modified graph requires $O(E)$ time, where $|E|$ is the number of edges. The overall time is then aggregated from the operations across all iterations, yielding a complexity of $O(|V| \times |S|^2 \times |E|)$. Similarly, to store the intermediate graph queues, if the number of nodes and edges are bounded by $O(|V|)$ and $O(|E|)$ respectively, the space complexity is $O(|V| \times |S| \times (|V| + |E|))$.

V. ONLINE PROCESSING

In the offline phase, PEPS undertakes optimizations specific to nUDFs. However, the online execution of prediction queries involves the combination of nUDFs and relational operators, leaving opportunities for further performance improvements.

We propose a two-step online optimization strategy before query execution, as shown in Fig. 8. Given a prediction query, PEPS parses it into a coarse-grained query plan, organized as a tree with relational operators and nUDFs as the nodes. We propose a two-step approach for online optimization as shown in Fig. 8. Given a prediction query, PEPS parses it into a coarse-grained query plan, organized as a tree with

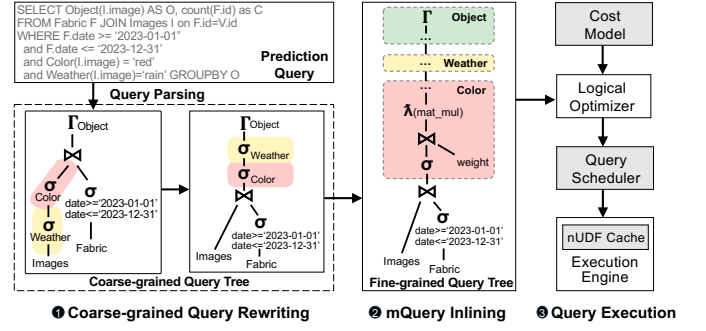


Fig. 8: Workflow of online prediction query processing.

relational operators and nUDFs as the nodes, before online execution. In the first stage, the system conducts *coarse-grained query rewriting* to reorganize the nodes within the query tree. Subsequently, in the second stage, the nUDFs are substituted with their corresponding mQueries, resulting in a fine-grained query plan composed of relational operators and UDOs. This procedure is referred to as *mQuery inlining*. These two stages constitute the heuristic query rewriting for prediction queries, culminating in an optimized query plan. It should be noted that the reason why we arrange the coarse-grained query rewriting as the first stage is that, the query rewriting may confront an inflated search space if the mQuery inlining is processed with raw plans, which is hard to unnest and optimize [7].

A. Coarse-grained Query Rewriting

This stage seeks to optimize prediction queries by employing heuristic rules to enhance efficiency while maintaining result equivalence. Specifically, an nUDF f can be nested within relational operators such as project and select, denoted as o_f . Given a prediction query PQ and a set $H = \{r_1, \dots, r_{|H|}\}$ of rules, each incorporating o_f . Coarse-grained query rewriting is to apply the rules to obtain a rewritten PQ^* , namely $PQ \xrightarrow{H} PQ^*$, such that (i) the executing results of PQ^* are equivalent to the results of PQ and (ii) the cost of executing PQ^* is smaller than executing PQ . We propose three novel rules with proofs, while other transformation rules in existing systems [41], [42] remain applicable:

Rule 1 (nUDF reordering): $o_f(R) \bowtie_p S \xrightarrow{r_1} o_f(R \bowtie_p S)$ with $r_1: sel(p) < \frac{1}{|S|}$, where $sel(p)$ is the selectivity of join predicate p on $R \bowtie S$, $sel(o_f)$ denotes the selectivity of o_f on R .

This reordering rule is applied when we can reduce the input size of nUDFs through operator reordering. An example can be seen in Fig. 8, consider a join between `Images` table and `Fabric` table where the join predicate on `id` produces a result size of $\frac{|Images|}{2}$ tuples. Since $sel(p) = \frac{1}{2 \cdot |Fabric|}$, the nUDF `Color()` is pulled up after the join to reduce the cost.

Proof: Let α and β represent the unit costs of the build and probe phases in hash joins. To prove the rule works, we should clarify that the cost after substitution, C_2 , should be smaller than the original cost, C_1 , namely, $C_2 - C_1 < 0$.

We know that $C_2 = C_f \cdot |R| \cdot |S| \cdot sel(p) + \alpha \cdot |R| + \beta \cdot |S|$ and $C_1 = C_f \cdot |R| + \alpha \cdot sel(o_f) \cdot |R| + \beta \cdot |S|$. Therefore we

have $C_2 - C_1 = |R| \cdot C_f \cdot [(sel(p) \cdot |S| - 1) + \frac{\alpha}{C_f} \cdot (1 - sel(o_f))]$. Since $0 \leq sel(o_f) \leq 1$ and $C_f \gg \alpha^1$, we could derive $\frac{\alpha}{C_f} \cdot (1 - sel(o_f)) \rightarrow 0$, which means the clause can be neglected in the equation. Therefore, $C_2 - C_1 < 0$ holds when $sel(p) < \frac{1}{|S|}$.

Rule 2 (Pairwise nUDF reordering): $o_{f_1}(R) \wedge o_{f_2}(R) \xrightarrow{r_2} o_{f_2}(R) \wedge o_{f_1}(R)$ with $r_2 : \frac{C_{f_2}}{1 - sel(o_{f_2})} < \frac{C_{f_1}}{1 - sel(o_{f_1})}$.

This rule applies to a prediction query involving multiple nUDF invocations in different parts (e.g., predicates and aggregations). As illustrated in Fig. 8, two nUDF predicates `Color()` and `Weather()` are applied to the `Images` table. The execution cost of `Weather()` is three times greater than that of `Color()`, while their selectivities remain comparable. Therefore, `Color()` should be executed first.

Proof: Given two nUDFs o_{f_1} and o_{f_2} executed sequentially on R . By default, we have the vanilla cost $C_1 = C_{f_1} \cdot |R| + C_{f_2} \cdot sel(o_{f_1}) \cdot |R|$. After rewriting, the cost changes to $C_2 = C_{f_2} \cdot |R| + C_{f_1} \cdot sel(o_{f_2}) \cdot |R|$. Therefore, we have $C_2 - C_1 = |R| \cdot [C_{f_2} \cdot (1 - sel(o_{f_1})) - C_{f_1} \cdot (1 - sel(o_{f_2}))]$. For $\frac{C_{f_2}}{1 - sel(o_{f_2})} < \frac{C_{f_1}}{1 - sel(o_{f_1})}$, $C_2 - C_1 < 0$ holds.

Rule 3 (nUDF decorrelation): $R \bowtie_{p \wedge o_f} S \xrightarrow{r_3} o_f(R \bowtie_p S)$ with $r_3 : sel(p) < \frac{1}{|S|}$.

For the joins comparing the results of o_f applied to R with values from S , this rule decouples o_f from the join predicates to avoid redundant comparisons.

Proof: Given a join with a regular predicate p and an nUDF-dependent predicate o_f regarding R . Let $C_1 = C_f \cdot |R| + \alpha \cdot |R| + \beta \cdot |S|$ be the original cost. When performing join on p first and then applying o_f to the joined results, the cost changes to $C_2 = C_f \cdot |R| \cdot |S| \cdot sel(p) + \alpha \cdot |R| + \beta \cdot |S|$. We have $C_2 - C_1 = C_f \cdot |R| \cdot (sel(p) \cdot |S| - 1)$. Therefore, $C_2 - C_1 < 0$ holds when $sel(p) < \frac{1}{|S|}$, proving the decorrelation is beneficial.

B. mQuery Inlining

After the nUDFs have been repositioned at a coarse level, we next substitute each nUDF f with its associated mQuery Q_f , which comprises a sequence of subqueries as defined in Definition 1. These subqueries implicitly expose the execution semantics of the model inference process (e.g., probability distribution) that conventional heuristic rules fail to utilize. Therefore, we propose two novel optimization strategies: (1) reordering operators between Q_f and other parts of $\mathcal{PQ}$ to pull up compute-intensive UDOs; (2) reusing UDOs across multiple inlined Q_f to reduce redundant computation.

Strategy 1 (UDO Pullup): $\theta(Q_f(R)) \rightarrow Q_f^\theta(R)$, where θ is one of the relational operators from the types of *aggregate*, *predicate*, and *join*, $Q_f^\theta = \langle o_1, \dots, o_k, \theta, o_{k+1}, \dots, o_m \rangle$, $o_i (1 \leq i \leq m \leq n)$ is an operator within Q_f and n is the number of operators in Q_f .

This strategy integrates θ into Q_f to defer the execution of expensive UDOs. By default, the execution of $\mathcal{PQ}$ re-

¹Empirically, for hash joins with millions of records, the build phase takes tens of milliseconds (α is estimated by microseconds), while executing DL inference (C_f) usually takes milliseconds.

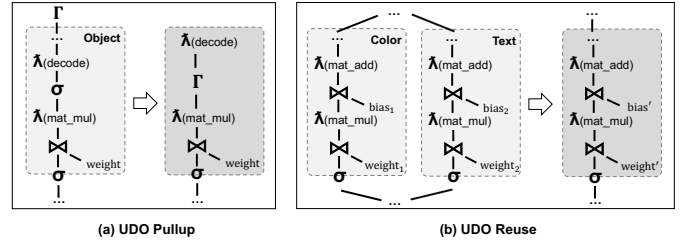


Fig. 9: Examples of the mQuery Inlining Strategies.

quires materializing the complete results of Q_f before applying θ . With the integration, specific UDOs are pulled up above θ in the query plan tree to prune the data tuples, reducing unnecessary UDO computations. As an example, consider the aggregation operator in Fig. 9a, namely `GROUPBY Object(image)`. Rather than aggregating string labels from the output of Q_f , PEPS incorporates the grouping hash computation directly into Q_f , using indices from probability distributions as hash keys. This avoids unnecessary label conversions and accelerates the entire process.

Strategy 2 (UDO Reuse): $\pi_{f_1 \vee f_2, \dots, \vee f_k} R \rightarrow Q_{fused}(R)$, where Q_{fused} is a sequence of subqueries merged from k mQueries.

For multiple nUDFs operating on the same input relation, this strategy is applied to identify and reuse UDOs across multiple mQueries. By using this strategy, the number of executions of the shared UDO is minimized, thereby reducing the overall processing time. Consider a query containing two nUDFs in Fig. 9b as an instance. During mQuery inlining, the corresponding two nUDFs are merged into one, thereby reducing the number of subqueries and intermediate results.

VI. EVALUATION

In this section, we first introduce the evaluation setup and then present a detailed analysis of the evaluation results.

A. Experimental Setup

The setup of our evaluation is listed as follows.

Hardware Configuration. We conducted our experiments on a machine running Ubuntu 16.04.7 LTS with an Intel(R) Xeon(R) Silver 4110 CPU @ 2.10GHz. The machine has 16 CPU cores and 128 GB of DDR4 DRAM main memory. For comparison with GPU, we equipped the machine with a Tesla V100.

Implementation Details. We implement PEPS by extending the modules of PostgreSQL. In particular: (i) we introduced the nUDF-related syntax to allow end users to launch prediction queries, as detailed in Section III; (ii) we defined and implemented 50+ UDOs in C using the MDA data model to support efficient DL computations; (iii) we utilized the PostgreSQL hooks system as a foundation to facilitate the nUDF creation, offline optimization, and online optimization of prediction queries. The plug-and-play extension model of the database allows PEPS to be easily integrated into an existing PostgreSQL instance without recompiling the code. Furthermore, PEPS can be seamlessly adapted to various

Q1 (nUDF Aggregation): <pre>SELECT COUNT(*), Type(image) O FROM images I INNER JOIN user_action U ON I.image_id = U.image_id WHERE U.action_type = 'like' AND I.upload_time >= '2023-01-01' GROUP BY O</pre>	Q2 (nUDF Filter): <pre>SELECT R.summary FROM images I INNER JOIN user_action U ON I.image_id = U.image_id INNER JOIN review R ON U.action_id = R.action_id WHERE Type(image) = 'cat' AND I.upload_time >= '2023-01-01' AND U.group_id = 2 AND U.action_type = 'comment'</pre>	Q3 (Multi-nUDF Filter): <pre>SELECT COUNT(I.image_id) FROM images I INNER JOIN user_action U ON I.image_id = U.image_id INNER JOIN review R ON U.action_id = R.action_id WHERE Type(image) = 'cat' AND U.action_type = 'comment' AND Rating(R.review_text) = 0</pre>	Q4 (Multi-nUDF Invocation): <pre>SELECT COUNT(action_id), Rating(reviews_text) Score, Spam(reviews_text) S FROM review GROUP BY Score, S</pre>	Q5 (nUDF-based Anomaly Detection): <pre>SELECT UA.user_id, AVG(Rating(R.review_text)), AVG(Spam(R.review_text)) FROM review R INNER JOIN user_action UA ON R.action_id = UA.action_id WHERE UA.user_id IN (SELECT DISTINCT U.user_id FROM images I INNER JOIN user_action U ON I.image_id = U.image_id AND U.action_type = 'like' INNER JOIN groups g ON Type(I.image) != g.group_type) GROUP BY UA.user_id</pre>
Q6 (Simple Query): <pre>SELECT Type(image) FROM images;</pre>	Q7 (Advanced nUDF Filter): <pre>SELECT G.group_name, COUNT(*) Type(image) O FROM images I INNER JOIN user_action U ON I.image_id = U.image_id INNER JOIN groups G ON U.group_id = G.group_id WHERE U.action_type = 'comment' AND I.upload_time >= '2023-01-01' AND GROUP BY G.group_name, O</pre>	Q8 (Advanced Multi-nUDF Filter): <pre>SELECT COUNT(I.image_id) FROM images I INNER JOIN user_action U ON I.image_id = U.image_id INNER JOIN review R ON U.action_id = R.action_id WHERE Type(image) = 'cat' AND AND Rating(R.review_text) < 3 AND Spam(R.review_text) = 0</pre>	Q9 (Advanced nUDF Aggregation): <pre>SELECT Rating(reviews_text) AVG(Spam(reviews_text)) spam_ratio FROM review GROUP BY Rating(review_text)</pre>	Q10 (nUDF-based Spam Checking): <pre>SELECT UA.user_id, AVG(Spam(R.review_text)) FROM review R INNER JOIN user_action UA ON R.action_id = UA.action_id WHERE UA.user_id IN (SELECT DISTINCT U.user_id FROM images I INNER JOIN user_action U ON I.image_id = U.image_id AND U.action_type = 'like' AND Type(I.image) = 'cat' AND UA.action_type = 'comment' GROUP BY UA.user_id HAVING AVG(Spam(R.review_text)) > 0.5</pre>

Fig. 10: Prediction queries used in the evaluation.

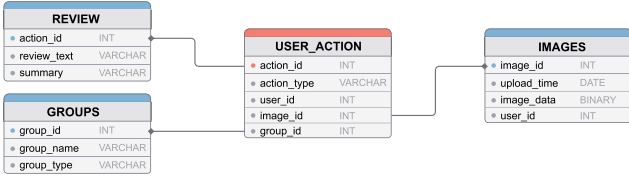


Fig. 11: Database schema.

database systems as its modular design is compatible with third-party extension interfaces provided by modern databases (e.g., ClickHouse [43] and DuckDB [44]).

Evaluation Metric. For system usability, we define *query synthesis time* as the CPU time elapsed from the model input until the corresponding mQuery generation. For system effectiveness, we measure both the prediction query processing time and the overall system processing time (including optimization). For economic efficiency, we use the estimated monetary costs based on AWS EC2 pricing models.

Models. The models used in the evaluation include TextCNN [45], ResNet [46], ResNeXt [26], and Inception-v3 [47]. TextCNN is a convolutional neural network (CNN) for text classification tasks, while the latter three models are all proposed for image classification tasks. ResNet is a deep CNN due to the residual connections. ResNeXt enhances the Inception model with a grouped convolution operator, and Inception-v3 is known for its high efficiency.

Analytical Datasets. To evaluate PEPS on real-world data analysis tasks, we design a database schema tailored for analyzing users on social media platforms and populate it with real-world data (Fig. 11). The *user_action* table records user activities on social media platforms, and the remaining tables store detailed information about post images, comments, and associated group profiles. To construct the tables containing unstructured data, we use four widely adopted datasets: MNIST [48], CIFAR10 [49], AGNews [50], and Amazon Product Review [51]. We utilize the first two image datasets,

selecting a subset for model training and randomly choosing a portion of the test set to insert into the *images* table for classification tasks. Similarly, we employ the latter two text datasets targeting topic labeling, spam detection, and sentiment analysis. The *user_action* table has 50,000 rows, *groups* has 1,000, and each remaining table has 10,000.

Evaluation Queries. To evaluate PEPS in processing prediction queries, our benchmark includes various query types and use cases. We employ the nUDF *Type()* for image categorization, *Rating()* for user scores, and *Spam()* for spam detection. By default, our benchmarks utilize ResNeXt-50 for image tasks and TextCNN for text tasks. Six representative queries are presented in Fig. 10. For more comprehensive queries, please refer to the technical report in our open-source repository [52].

Baselines. We compare **PEPS** with both *loose integration* methods and *tight integration* methods. For loose integration methods, we adopt two UDF-based approaches: PyTorch [53] with Just-In-Time compilation (**JIT**), and ONNX Runtime (**ONNX**) [36]. Each approach is evaluated under CPU-only and GPU-accelerated configurations (**JIT(GPU)**, **ONNX(GPU)**). We also include **MADlib** [5], a loose integration solution based on TensorFlow. For tight integration, we utilize **DL2SQL** [7] and **NN2SQL** [6]. DL2SQL translates DL operations into SQL queries using a relational representation. NN2SQL suggests using array data types, but only supports simple neural networks without complex DL operations. To highlight the influence of optimization techniques in PEPS, we provide **PEPS(w/o op)** which directly translates DL models into mQueries without any optimizations.

B. Simple Prediction Query Performance

In this subsection, we aim to demonstrate the performance of all different approaches for in-DB DL inference. Therefore, we select Q6, a simple prediction query for testing, which

TABLE III: Overall evaluation of different model prediction queries in terms of response time (seconds).

Models	DL2SQL	MADlib	ONNX	ONNX(GPU)	JIT	JIT(GPU)	PEPS(w/o op)	PEPS	Maximum Improvement
ResNet-18	28.518	9.176	5.697	2.226	2.436	2.532	1.453	1.194	23.88×
ResNeXt-29	45.614	11.782	2.778	2.362	2.283	2.181	2.031	1.606	28.40×
ResNeXt-50	60.785	14.927	4.558	2.866	3.321	3.381	2.693	2.238	27.16×
Inception-v3	100.774	16.564	14.158	3.717	11.255	3.062	9.197	7.243	13.91×
TextCNN	6.469	7.166	0.267	0.361	1.098	1.576	0.185	0.101	64.05×

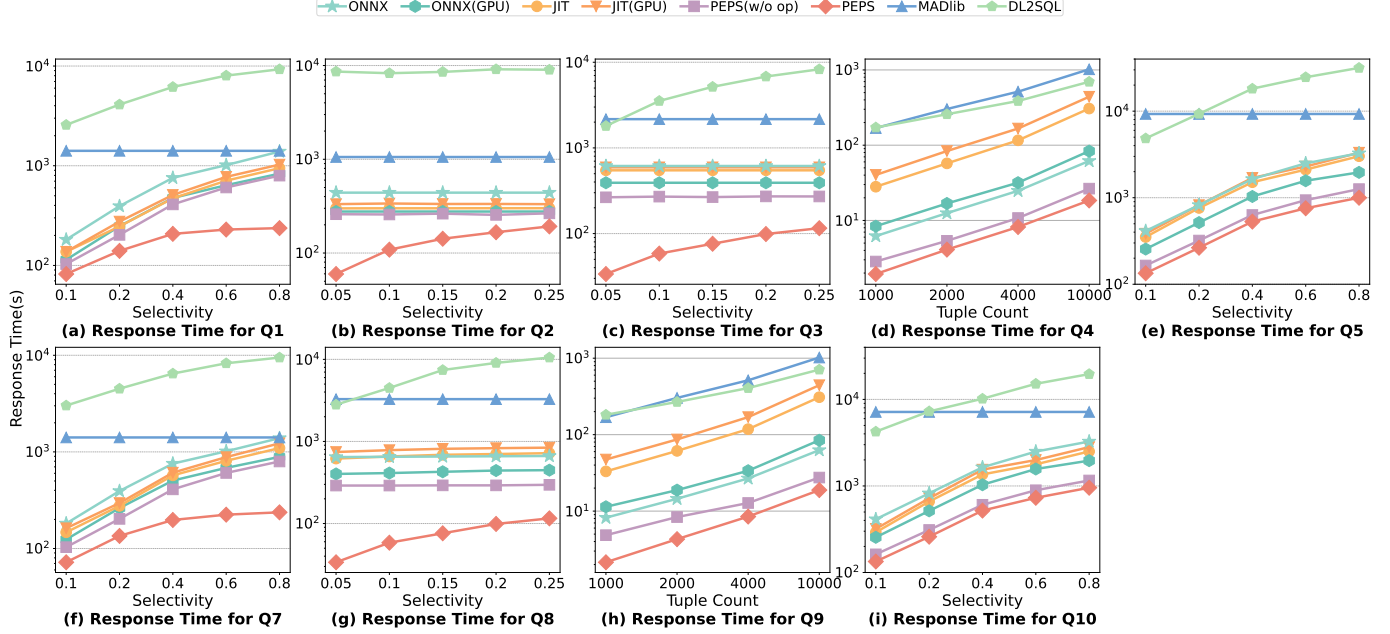


Fig. 12: Efficiency evaluation of online optimization in terms of response time for complex prediction queries.

directly performs model inference and excludes other relational operators that other baselines may not support. We use a default batch size of 64 in these experiments.

Table III shows the main results. Overall, PEPS exhibits significant advantages for in-DB DL inference compared with CPU-based implementations, and achieves a $1.42\text{--}64.05\times$ speedup. Among the baselines, DL2SQL performs the worst, illustrating the inefficiency of the relational data model in executing large-scale linear algebra computations. Meanwhile, PEPS outperforms MADlib, ONNX, and JIT by $18.98\times$, $2.63\times$, and $3.47\times$, respectively. Compared with GPU-based baselines, PEPS is slower than ONNX(GPU) and JIT(GPU) by $1.95\times$ and $2.37\times$ on Inception-v3. This is due to the superior parallel computing capabilities of GPUs for deeper and more complex architectures. However, for other models, PEPS consistently outperforms these two methods by $1.28\text{--}15.06\times$. This stems from the overhead of data transfer between CPU-GPU memory and cross-process communication, which dominates execution time for less complex models. We also observe that PEPS significantly reduces inference time compared to PEPS(w/o op). It achieved performance improvements of 17.8%, 20.9%, 16.9%, 21.2%, and 45.4% on the five models, demonstrating the effectiveness of our offline optimization techniques.

C. Complex Prediction Query Performance

In this subsection, we show the performance results for complex prediction queries (Q1-Q5). Fig. 12 shows the response time for our full benchmark suite. For Q4, we vary the size of the *review* table. For the remaining queries, we vary the join selectivity between tables. As baselines, we show the results of loose integration methods and DL2SQL. We discuss the evaluation for each query type in detail below.

1) *Q1(nUDF Aggregation)*: This query aggregates user likes across different image categories, with join selectivity influencing the number of resulting tuples. As shown in Fig. 12a, all methods except MADlib, exhibit increased execution time under higher selectivity. This is due to the default query plan where nUDF computations are performed after table joins. In contrast, MADlib materializes complete inference results before processing other relational operations. Through our query rewriting technique, nUDF computations are pushed down with fewer inputs, thereby reducing execution times. At the highest selectivity level, applying our optimization techniques resulted in a speedup of $3.52\times$ to $39.16\times$.

2) *Q2(nUDF Filter)*: This query employs nUDFs as filters to retrieve user evaluations for specific categories. We demonstrate the advantages of achieving up to a $17.7\times$ speedup compared to loose integration approaches. In this type, nUDFs are typically computed and applied as filters immediately after a table scan, making conventional methods insensitive to the

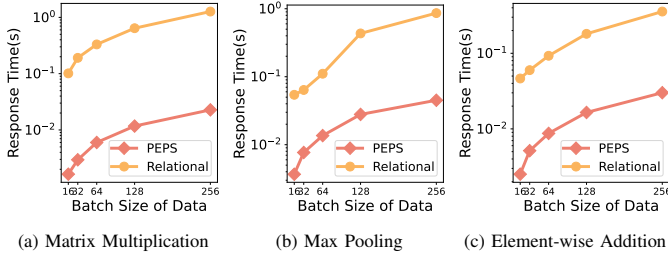


Fig. 13: Performance of UDOs in PEPS.

join selectivity. However, as selectivity gradually decreases, pulling nUDF predicates above the join operation can reduce the inputs to nUDF, thereby yielding performance benefits.

3) *Q3(Multi-nUDF Filter)*: We add `Rating()` as an additional filter based on `Q2`, which incurs significantly lower cost than `Type()`. The response time result is shown in Fig. 12c. Overall, the performance trends of the baseline methods align with those observed in `Q2`. In specific, PEPS achieves notable performance improvements, ranging from $3.39\times$ to $71.6\times$. This is due to two factors: delayed execution of nUDFs and evaluation reordering of two nUDFs. By cost analysis, PEPS prioritizes the computation of `Rating()`, enabling early filtering of tuples at a minimal cost.

4) *Q4(Multi-nUDF Invocation)*: This query focuses on invoking multiple nUDFs on a single table. As shown in Fig. 12d, the response time generally increases linearly with the increasing number of input tuples. Compared with baselines, PEPS(w/o op) has a speedup of $2.18\text{--}38.5\times$. Moreover, leveraging *UDO reuse* in Section V-B, the online optimization can further achieve a $1.5\times$ performance improvement.

5) *Q5(nUDF-based Anomaly Detection)*: This complex query combines multiple nUDFs in both join conditions and projections to detect anomalous user behavior. As shown in Fig. 12e, PEPS achieves a $1.9\text{--}69.2\times$ speedup over the baselines, driven by both effective reordering of nUDFs and the merging of multiple nUDF projections.

D. Tight Integration Performance

We further evaluate the performance of PEPS compared to tight integration methods. Since NN2SQL is limited to simple neural networks with a single hidden layer [6], we conduct our performance evaluation under this setting. Fig. 14 shows the response time for model inference on the MNIST dataset, varying the hidden layer size. We observe that DL2SQL’s relational representation significantly underperforms the array-based representation, primarily due to increased memory consumption from explicit indices and poor locality. Besides, PEPS consistently surpasses NN2SQL across all sizes of hidden layers, with speedups ranging from $1.6\times$ to $2.6\times$. This confirms the performance optimization of our underlying UDO implementation. Our tight integration design not only excels in basic matrix operations but also assembles complex DL models, demonstrating strong scalability.

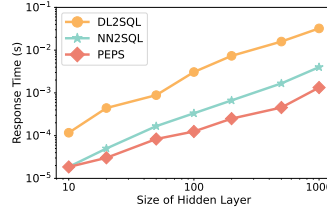


Fig. 14: Comparison between tight integration methods.

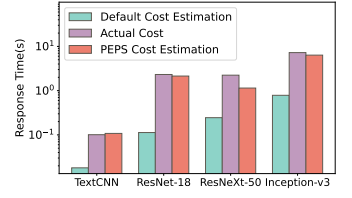
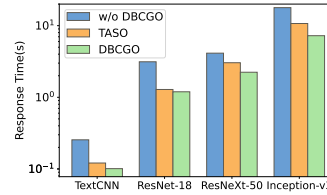
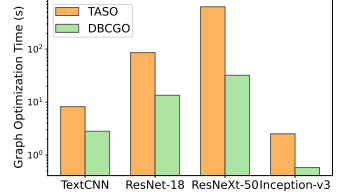


Fig. 15: Comparison between different cost models.



(a) Effects of the DBCGO algorithm.



(b) Overhead of graph optimization algorithm

Fig. 16: Efficiency evaluation of Offline Optimization in PEPS.

E. Component Analysis

Efficiency of UDOs. We compare three representative UDOs in PEPS (*mat_mul*, *max_pool*, and *eadd*) against their equivalent relational implementations on CIFAR10 dataset with varying batch sizes. As shown in Fig. 13, PEPS achieves a $58.5\times$ speedup in matrix multiplication, and is $13.2\times$ and $12.7\times$ faster in max pooling and element-wise addition. These results justify our decision to implement specialized UDOs as fundamental primitives for prediction query processing.

Efficiency of Offline Optimization. To evaluate the effectiveness of offline optimization, we first examine the cost model in the DBCGO algorithm. We compare the default database cost model, our customized cost model, and actual running costs for `Q6` using four different models, as shown in Fig. 15. We normalize the cost into the running time based on the ratio $r = \frac{\text{seq_time}}{\text{seq_scan_cost}}$. As the default optimizer is unable to assess the cost of UDOs in mQueries, it underestimates by up to $13\times$ compared to the actual cost. In contrast, our cost model provides estimates that are much closer to the ground truth but still underestimates by 9% to 30%. This is primarily due to our cost model’s optimistic assumptions about the cost of parallel processing, aligning with the theoretical lower bound we proposed.

We further compare DBCGO with TASO [39], an open-sourced graph optimization method that requires manual optimization of DL models before inputting them into PEPS. As shown in Fig. 16a, both approaches improve in-DB inference. DBCGO achieves a maximum $2.6\times$ speedup, outperforming TASO by up to $1.5\times$, owing to its DBMS-specific cost model in Section IV-B2. While the gains are relatively small, DBCGO is deeply integrated into the databases and can rapidly produce optimization results based on cost estimation instead of time-consuming hardware execution evaluation. Fig. 16b quantifies the optimization overhead, where DBCGO exhibits a $2.93\text{--}19.7\times$ saving in optimization time. By integrating

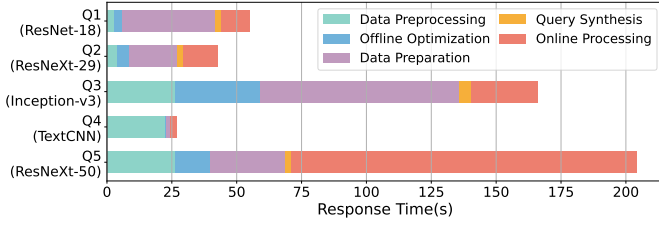


Fig. 17: Overhead breakdown of PEPS components.
TABLE IV: Contribution of optimizations in PEPS

Offline Opt.	Online Opt.	Response Time ($\uparrow$ Speedup)
$\times$	$\times$	288.41s
$\checkmark$	$\times$	231.35s ($\uparrow 1.25\times$)
$\times$	$\checkmark$	33.77s ($\uparrow 8.54\times$)
$\checkmark$	$\checkmark$	25.63s ($\uparrow 11.25\times$)

DBC GO, PEPS eliminates manual optimization while achieving better effectiveness and efficiency.

Overhead Breakdown. To analyze the overhead of various components of the system, we conduct a complete end-to-end measurement across Q1-Q5, including the time spent on storing unstructured data (data preprocessing), offline optimization, storing model parameters as tables (data preparation), query synthesis, and online processing. We employ different models for the nUDFs in each workload to assess the impact of the model on various components. Fig. 17 shows the results of the overhead breakdown. Specifically, the average breakdown is as follows: data preprocessing (25.39%), offline optimization (8.86%), data preparation (34.64%), query synthesis (3.07%), and online processing (28.05%). Although the offline components contribute substantial overhead, the primary cost stems from writing new array tables. This can be mitigated through parallel batch writing and stream processing, which are orthogonal to the focus of our work.

Contribution of Optimizations. To quantify the effects of our optimization techniques, we conduct an ablation study using Inception-v3 under Q3. Table V presents the results of different optimization combinations. Compared to PEPS without any optimization, offline optimization brings a $1.25\times$ speedup, while online optimization achieves an $8.54\times$ speedup. Enabling both optimization techniques, PEPS achieves the highest performance gain by $11.25\times$.

F. Usability of PEPS

We evaluate the complexity of automatic prediction query synthesis. The query synthesis overhead and model details are summarized in Table V, where the time includes the total duration for loading the model into PEPS, performing offline optimization, generating corresponding mQueries, and creating the necessary model weight tables. As the model complexity grows and accuracy increases, the synthesis time increases due to the larger search space in graph optimization and the creation of additional weight tables. Nonetheless, even for the most complex model, the query synthesis time is 45s.

TABLE V: Query Synthesis Results for DL Models.

Model	no. of DL operators	Accuracy	Lines of mQueries Synthesized	Query Synthesis Time (s)
TextCNN	15	79.87%	34	0.91
ResNet-18	50	92.34%	198	7.18
ResNeXt-50	86	93.89%	375	20.24
Inception-v3	138	94.77%	606	45.26

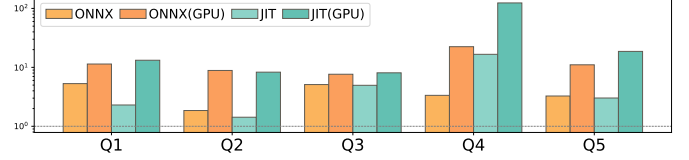


Fig. 18: Comparison of economic cost.

Compared to the hours of query composition process by users, this represents a significant reduction in user burden.

G. Economic Cost Analysis

We further evaluate PEPS from an economic perspective using AWS EC2 pricing models. Our hardware configuration matches the c6g.4xlarge service priced at \$0.5440 per hour. With an additional V100 GPU, its monetary cost is measured by \$3.06 per hour according to the p3.2xlarge service. We compare PEPS with JIT, ONNX, and their GPU-based versions across five workloads, using the same configurations in Fig. 17. Fig. 18 shows the relative economic cost of these methods compared to PEPS. Although GPU acceleration provides noticeable performance improvements, their high cost leads to an economic expense of $7.6\times$ to $123.5\times$ greater than PEPS. Therefore, for users with limited budgets, PEPS offers significant advantages over GPU-based loose integrations.

VII. RELATED WORK

In-Database DL inference. In-DB DL inference has become a hot topic in recent years. Madlib [5], SystemDS [54], Vertica-ML [14], and DB4ML [55] are classical loose integration methods that combine the deployment of DL models through UDFs. Supported by cutting-edge DL systems and specialized hardware, AI-centric loose integrations [17]–[21], have further enhanced the speed of in-DB inference, with a motivation to move the DBMS engine from a relational to a tensor abstraction. In contrast, approaches such as [7], [56], [57] implement inference computations as SQL queries, thereby minimizing disruptions to the DBMS and opening substantial opportunities for performance optimization. Our work aligns with this tight integration paradigm, striving to bridge the performance gap while maintaining high usability.

UDF optimization. Recent UDF optimization techniques span three main strategies: UDF-to-SQL translation [2], [58], [59], UDF-to-IR frameworks [60], [61], and in-engine integration leveraging JIT compilation and vectorization [62], [63]. Moreover, YeSQL [63] employs a compiler-based approach to inline Python UDFs, and QFusor [64] further co-optimizes

these inlined UDFs within query plans. However, these methods target procedural UDFs with control flow and loops, which are fundamentally different from DL inference that relies on tensor computations via specialized DL frameworks. To bridge this gap, we introduce nUDFs, focusing on optimizations based on model computation graphs and DL operators rather than traditional code compilation techniques. Our approach complements existing techniques by providing specialized support for in-DB DL inference.

DL optimization. DL optimizations span graph-level and operator-level approaches. Graph-level optimizations, explored in [29], [39], [65], [66], focus on computation graph transformations to minimize memory and execution time, largely employing heuristic searches with underlying computational complexity. As modern GPUs provide greater potential for high-performance parallel computations, Nimble [40], IOS [67], and Autograph [30] further take the parallel paradigm in graph optimizations. On the other side, operator-level optimization aims at accelerating individual operators within DL models, leveraging libraries such as cuBLAS.

VIII. CONCLUSION

In this paper, we focus on building a user-friendly and efficient prediction query processing system PEPS. The system is specifically designed with extended SQL syntax for model storage and tensor data management. To support in-DB DL inference, it automates the process of converting models into a series of relational operations and dedicated UDOs. To achieve superior performance, PEPS introduces an innovative model graph optimization algorithm tailored for DBMS and employs heuristic query rewriting. Our experiments demonstrate that PEPS provides significant improvements on usability, economic costs, and prediction query performance against all the baselines. As we continue to develop PEPS, we aim to further optimize its execution for highly complex architectures in the near future, where substantial parallel branches present unique scheduling challenges. We also plan to support more types of the model called by prediction queries, such as transformers, and extend the ability to existing approaches like NN2SQL.

REFERENCES

- [1] C. R. Aberger, A. Lamb, K. Olukotun, and C. Ré, “Levelheaded: A unified engine for business intelligence and linear algebra querying,” in *Proceedings of the IEEE International Conference on Data Engineering*, 2018, pp. 449–460.
- [2] J. V. D’silva, F. De Moor, and B. Kemme, “Aida: abstraction for advanced in-database analytics,” *PVLDB*, p. 1400–1413, 2018.
- [3] V. Sanca and A. Ailamaki, “Analytical engines with context-rich processing: Towards efficient next-generation analytics,” in *Proceedings of the IEEE International Conference on Data Engineering*, 2023, pp. 3699–3707.
- [4] M. Stonebraker, P. Brown, D. Zhang, and J. Becla, “Scidb: A database management system for applications with complex analytics,” *Computing in Science and Engineering*, vol. 15, no. 3, pp. 54–62, 2013.
- [5] J. M. Hellerstein, C. Ré, F. Schoppmann, D. Z. Wang, E. Fratkin, A. Gorajek, K. S. Ng, C. Welton, X. Feng, K. Li, and A. Kumar, “The madlib analytics library: or mad skills, the sql,” *PVLDB*, p. 1700–1711, 2012.
- [6] M. E. Schüle, A. Kemper, and T. Neumann, “NN2SQL: let SQL think for neural networks,” in *BTW*, vol. P-331, 2023, pp. 183–194.
- [7] Q. Lin, S. Wu, J. Zhao, J. Dai, F. Li, and G. Chen, “A comparative study of in-database inference approaches,” in *Proceedings of the IEEE International Conference on Data Engineering*, 2022, pp. 1794–1807.
- [8] M. Paganelli, P. Sottovia, K. Park, M. Interlandi, and F. Guerra, “Pushing ml predictions into dbms,” *TKDE*, pp. 10 295–10 308, 2023.
- [9] “Model serving with databricks,” <https://docs.databricks.com/en/machine-learning/model-serving/index.html>, 2024.
- [10] “Model inference overview,” <https://cloud.google.com/bigquery/docs/inference-overview>, 2024.
- [11] “Polardb for ai,” <https://www.alibabacloud.com/help/en/polardb/polardb-for-mysql/user-guide/polardb-for-ai/>, 2023.
- [12] Y. Peng, K. Chen, L. Shou, D. Jiang, and G. Chen, “AQUA: automatic collaborative query processing in analytical database,” *PVLDB*, vol. 16, no. 12, pp. 4006–4009, 2023.
- [13] M. Boehm, M. W. Dusenberry, D. Eriksson, A. V. Evfimievski, F. M. Manshadi, N. Pansare, B. Reinwald, F. R. Reiss, P. Sen, A. C. Surve, and S. Tatikonda, “Systemml: declarative machine learning on spark,” *PVLDB*, p. 1425–1436, 2016.
- [14] A. Fard, A. Le, G. Larionov, W. Dhillon, and C. Bear, “Vertica-ml: Distributed machine learning in vertica database,” in *SIGMOD*, 2020, p. 755–768.
- [15] A. S. Foundation, “Apache arrow,” <https://arrow.apache.org/>, accessed: 2024.
- [16] —, “Apache parquet,” <https://parquet.apache.org/>, accessed: 2024.
- [17] D. He, S. C. Nakandala, D. Banda, R. Sen, K. Saur, K. Park, C. Curino, J. Camacho-Rodríguez, K. Karanasos, and M. Interlandi, “Query processing on tensor computation runtimes,” *PVLDB*, p. 2811–2825, 2022.
- [18] A. Gandhi, Y. Asada, V. Fu, A. Gemawat, L. Zhang, R. Sen, C. Curino, J. Camacho-Rodríguez, and M. Interlandi, “The tensor data platform: Towards an ai-centric database system,” in *CIDR*, 2023.
- [19] Y.-C. Hu, Y. Li, and H.-W. Tseng, “Tcudb: Accelerating database with tensor processors,” in *SIGMOD*, 2022, p. 1360–1374.
- [20] Y. Asada, V. Fu, A. Gandhi, A. Gemawat, L. Zhang, D. He, V. Gupta, E. Nosakhare, D. Banda, R. Sen, and M. Interlandi, “Share the tensor tea: how databases can leverage the machine learning ecosystem,” *PVLDB*, p. 3598–3601, 2022.
- [21] M. Schleich, A. Shaikhha, and D. Suciu, “Optimizing tensor programs on flexible storage,” 2023.
- [22] H. Zhang, Y. Liu, and J. Yan, “Cost-intelligent data analytics in the cloud,” in *CIDR*, 2024.
- [23] D. Hirn and T. Grust, “One WITH RECURSIVE is worth many gotos,” in *Proceedings of The International Conference on Management of Data*, 2021, pp. 723–735.
- [24] M. Blacher, J. Giesen, S. Laue, J. Klaus, and V. Leis, “Machine learning, linear algebra, and more: Is SQL all you need?” in *CIDR*, 2022.
- [25] Q. Lin, S. Wu, J. Zhao, J. Dai, M. Shi, G. Chen, and F. Li, “Smartlite: A dbms-based serving system for DNN inference in resource-constrained environments,” *PVLDB*, vol. 17, no. 3, pp. 278–291, 2023.
- [26] S. Xie, R. B. Girshick, P. Dollár, Z. Tu, and K. He, “Aggregated residual transformations for deep neural networks,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2017, pp. 5987–5995.
- [27] C. Brücke, P. Härtling, R. E. Palacios, H. Patel, and T. Rabl, “Tpcx-ai - an industry standard benchmark for artificial intelligence and machine learning systems,” *PVLDB*, vol. 16, no. 12, pp. 3649–3661, 2023.
- [28] J. M. Hellerstein, “Practical predicate placement,” in *Proceedings of The International Conference on Management of Data*, 1994, pp. 325–335.
- [29] J. Fang, Y. Shen, Y. Wang, and L. Chen, “Optimizing dnn computation graph using graph substitutions,” *PVLDB*, p. 2734–2746, 2020.
- [30] Y. Zhao, Q. Sun, Z. He, Y. Bai, and B. Yu, “Autograph: Optimizing DNN computation graph for parallel GPU kernel execution,” in *Proceedings of The AAAI Conference on Artificial Intelligence*, 2023, pp. 11 354–11 362.
- [31] S. Luo, Z. J. Gao, M. Gubanov, L. L. Perez, and C. Jermaine, “Scalable linear algebra on a relational database system,” *TKDE*, pp. 1224–1238, 2019.
- [32] B. Yuan, D. Jankov, J. Zou, Y. Tang, D. Bourgeois, and C. Jermaine, “Tensor relational algebra for distributed machine learning system design,” *PVLDB*, p. 1338–1350, 2021.
- [33] O. Dolmatova, N. Augsten, and M. H. Böhlen, “A relational matrix algebra and its implementation in a column store,” in *SIGMOD*, 2020, p. 2573–2587.

- [34] S. E. Whang, Y. Roh, H. Song, and J. Lee, "Data collection and quality challenges in deep learning: a data-centric AI perspective," *VLDB J.*, vol. 32, no. 4, pp. 791–813, 2023.
- [35] SQL:2023, "Sql:2023 standard. database languages–sql–part 2: Foundation," ISO/IEC 9075-2:2023.
- [36] "Onnx runtime," <https://www.onnxruntime.ai/>, 2024.
- [37] M. Sichert and T. Neumann, "User-defined operators: Efficiently integrating custom algorithms into modern databases," *PVLDB*, vol. 15, no. 5, pp. 1119–1131, 2022.
- [38] F. Geerts, T. Muñoz, C. Riveros, J. V. den Bussche, and D. Vrgoc, "Matrix query languages," *SIGMOD Rec.*, vol. 50, no. 3, pp. 6–19, 2021.
- [39] Z. Jia, O. Padon, J. Thomas, T. Warszawski, M. Zaharia, and A. Aiken, "TASO: optimizing deep learning computation with automatic generation of graph substitutions," in *Proceedings of The ACM Symposium on Operating Systems Principles*, 2019, pp. 47–62.
- [40] W. Kwon, G. Yu, E. Jeong, and B. Chun, "Nimble: Lightweight and parallel GPU task scheduling for deep learning," in *Proceedings of The Advances in Neural Information Processing Systems*, 2020, pp. 8343–8354.
- [41] C. A. Galindo-Legaria and M. Joshi, "Orthogonal optimization of subqueries and aggregation," in *SIGMOD*, 2001, pp. 571–581.
- [42] P. Fent, A. Birler, and T. Neumann, "Practical planning and execution of groupjoin and nested aggregates," *VLDB J.*, vol. 32, no. 6, pp. 1165–1190, 2023.
- [43] R. Schulze, T. Schreiber, I. Yatsishin, R. Dahimene, and A. Milovidov, "Clickhouse - lightning fast analytics for everyone," *PVLDB*, vol. 17, no. 12, pp. 3731–3744, 2024.
- [44] R. Atwal, P. A. Boncz, R. Boyd, A. Courtney, T. Döhmen, F. Gerlinghoff, J. Huang, J. Hwang, R. Hyde, E. Felder *et al.*, "Motherduck: Duckdb in the cloud and in the client," in *CIDR*, 2024.
- [45] Y. Kim, "Convolutional neural networks for sentence classification," in *EMNLP*, 2014, pp. 1746–1751.
- [46] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 770–778.
- [47] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the inception architecture for computer vision," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 2818–2826.
- [48] L. Deng, "The mnist database of handwritten digit images for machine learning research," *IEEE Signal Processing Magazine*, vol. 29, no. 6, pp. 141–142, 2012.
- [49] A. Krizhevsky, "Learning multiple layers of features from tiny images," Tech. Rep., 2009.
- [50] X. Zhang, J. Zhao, and Y. LeCun, "Character-level convolutional networks for text classification," in *Proceedings of The Advances in Neural Information Processing Systems*, 2015, pp. 649–657.
- [51] R. He and J. J. McAuley, "Ups and downs: Modeling the visual evolution of fashion trends with one-class collaborative filtering," in *Proceedings of the International Conference on World Wide Web*, 2016, pp. 507–517.
- [52] "Technical report," <https://github.com/zjuDBSystems/PEPS>, 2025.
- [53] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimeshein, L. Antiga, A. Desmaison, A. Köpf, E. Z. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "Pytorch: An imperative style, high-performance deep learning library," in *Proceedings of The Advances in Neural Information Processing Systems*, 2019, pp. 8024–8035.
- [54] M. Boehm, I. Antonov, S. Baunsgaard, M. Dokter, R. Ginthör, K. In-nerebner, F. Klezin, S. N. Lindstaedt, A. Phani, B. Rath, B. Reinwald, S. Siddiqui, and S. B. Wrede, "Systemds: A declarative machine learning system for the end-to-end data science lifecycle," in *CIDR*, 2020.
- [55] M. Jasny, T. Ziegler, T. Kraska, U. Röhm, and C. Binnig, "DB4ML - an in-memory database kernel with machine learning support," in *SIGMOD*, 2020, pp. 159–173.
- [56] M. A. Khamis, H. Q. Ngo, X. Nguyen, D. Olteanu, and M. Schleich, "AC/DC: in-database learning thunderstruck," in *DEEM@SIGMOD*, 2018, pp. 8:1–8:10.
- [57] M. Schleich, D. Olteanu, M. Abo Khamis, H. Q. Ngo, and X. Nguyen, "A layered aggregate engine for analytics workloads," in *SIGMOD*, 2019, p. 1642–1659.
- [58] K. Ramachandra, K. Park, K. V. Emani, A. Halverson, C. A. Galindo-Legaria, and C. Cunningham, "Froid: Optimization of imperative programs in a relational database," *PVLDB*, vol. 11, no. 4, pp. 432–444, 2017.
- [59] S. Gupta, S. Purandare, and K. Ramachandra, "Aggify: Lifting the curse of cursor loops using custom aggregates," in *SIGMOD*, 2020, pp. 559–573.
- [60] S. Palkar, J. Thomas, D. Narayanan, P. Thaker, R. Palamuttam, P. Negi, A. Shanbhag, M. Schwarzkopf, H. Pirk, S. P. Amarasinghe, S. Madden, and M. Zaharia, "Evaluating end-to-end optimization for data analytics applications in weld," *PVLDB*, vol. 11, no. 9, pp. 1002–1015, 2018.
- [61] K. Karanasos, M. Interlandi, F. Psallidas, R. Sen, K. Park, I. Popivanov, D. Xin, S. Nakandala, S. Krishnan, M. Weimer, Y. Yu, R. Ramakrishnan, and C. Curino, "Extending relational query processing with ML inference," in *CIDR*, 2020.
- [62] M. Raasveldt and H. Mühleisen, "Vectorized udfs in column-stores," in *SSDBM*, 2016, pp. 16:1–16:12.
- [63] Y. Fofoulas, A. Simitsis, L. Stamatogiannakis, and Y. Ioannidis, "Yesql: 'you extend sql' with rich and highly performant user-defined functions in relational databases," *PVLDB*, p. 2270–2283, 2022.
- [64] K. Chasialis, T. Palaiologou, Y. Fofoulas, A. Simitsis, and Y. E. Ioannidis, "Qfusor: A UDF optimizer plugin for SQL databases," in *Proceedings of the IEEE International Conference on Data Engineering*, IEEE, 2024, pp. 5457–5460.
- [65] T. Chen, T. Moreau, Z. Jiang, L. Zheng, E. Q. Yan, H. Shen, M. Cowan, L. Wang, Y. Hu, L. Ceze, C. Guestrin, and A. Krishnamurthy, "TVM: an automated end-to-end optimizing compiler for deep learning," in *OSDI*, 2018, pp. 578–594.
- [66] "Nvidia tensorrt," <https://developer.nvidia.com/tensorrt>, accessed: 2024.
- [67] Y. Ding, L. Zhu, Z. Jia, G. Pekhimenko, and S. Han, "IOS: inter-operator scheduler for CNN acceleration," in *Proceedings of Machine Learning and Systems*, 2021.